

Embedded systems I / embedded programming

The interrupts

Pascal Sartoretti

v2.0

Interrupts : Objectives

Working with interruptions (Interrupts)



Interrupts
- **Why**
- **General implementation**

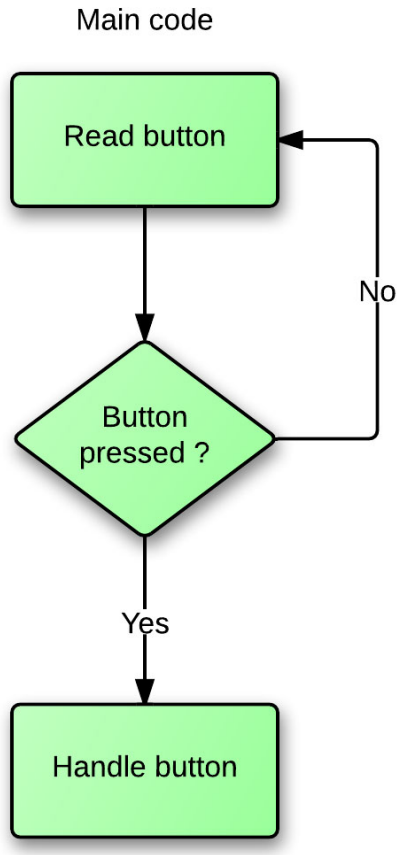
CubeMX configuration

Multi-thread caution

Sleep usage

Interrupts : Problem without interrupts

Reacts with a button pressed



Until now

- Pool button status
- If not pressed, loop
- When pressed, handle button job

Problems

- Can’t do anything else
- Waste of time and energy

Interrupts : The solution

Reacts with a button pressed with interrupts

Do what you need in main program

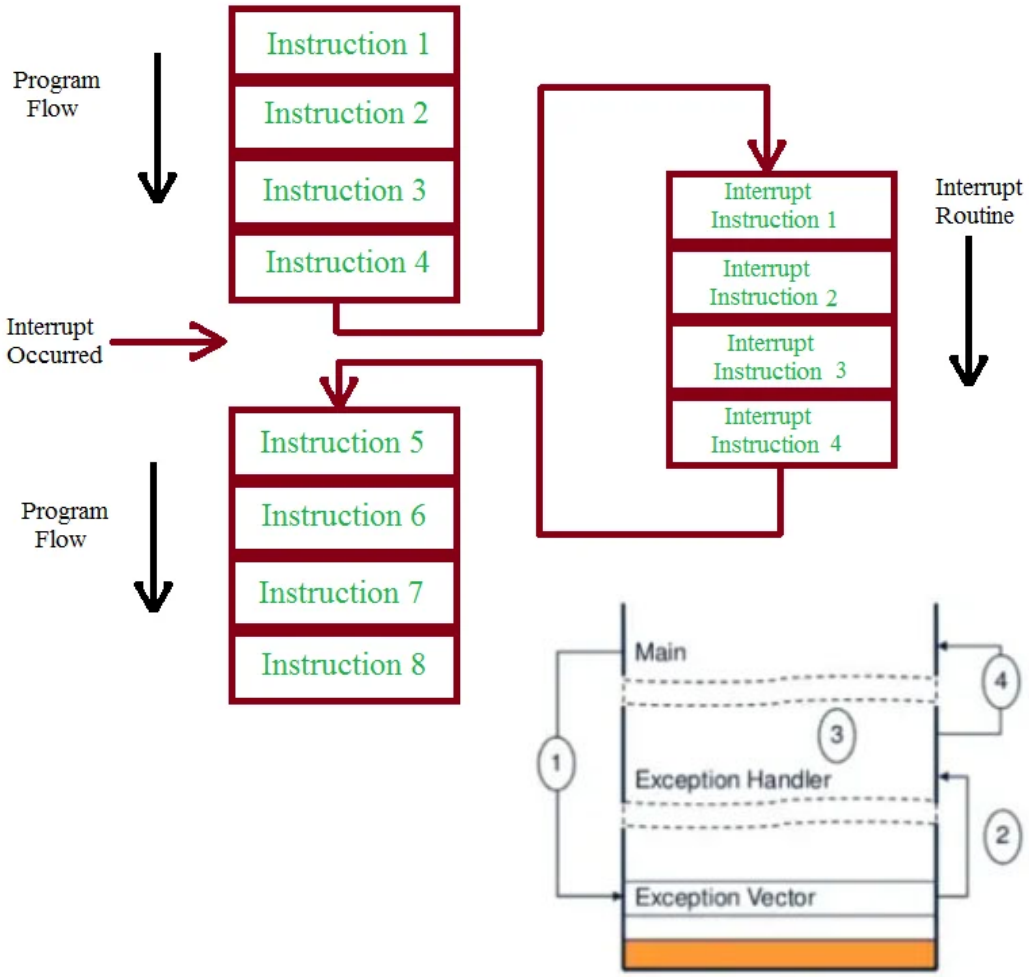
- **Do any job or**
- **Sleep when nothing to do**

When button is pressed (wake from sleep if needed)

- **Program flow in interrupted, processor jumps to a specific location**
- **When button process is over, resume activity where it was**

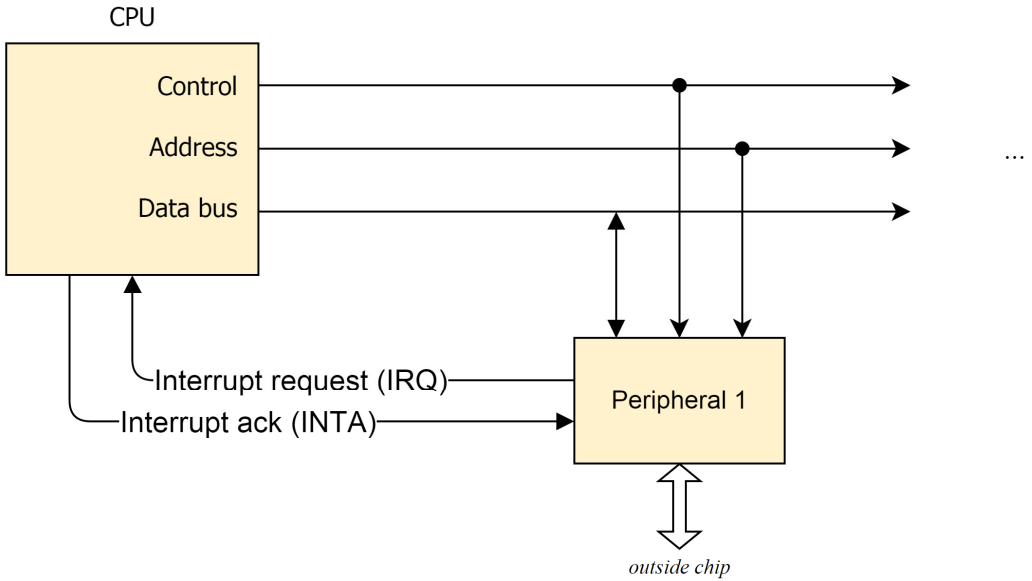
Interrupts : The solution

- main program runs normally
- When button is pressed Program flow in interrupt, processor jumps to a specific location
- When button process is over, resume activity where it was



Interrupts : The condition to occurs

- The event we want to monitor should be able to interrupt the processor
- The user must enable corresponding interrupts

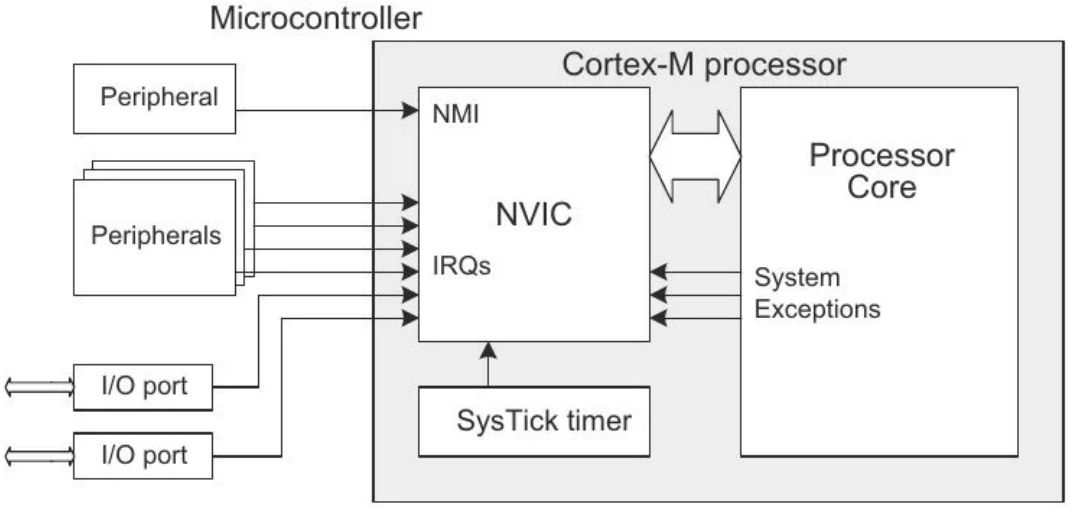


Interrupts : Characteristics

- Interrupts can be disabled (masked)
- Peripherals are interrupt sources (example)
  -
  -
  -
- Changes the program flow like a function call
  - Happen asynchronously, anywhere

Interrupts : STM32 view

- 140 interrupts sources in STM32U5G9
- 15 priority level



Exception source	Priority level	
Reset	-3	Fixed hardcoded priority
Non-Maskable Interrupt (NMI)	-2	
Hard Fault	-1	
Other exceptions including: - **Peripheral interrupts** - Software exceptions	Programmable level from 0 to 15	

Interrupts : Caution with main

main and interrupt are two “threads”

When main() and interrupt needs to share, for example, the same hardware, the interrupts must be masked in main when this hardware is used

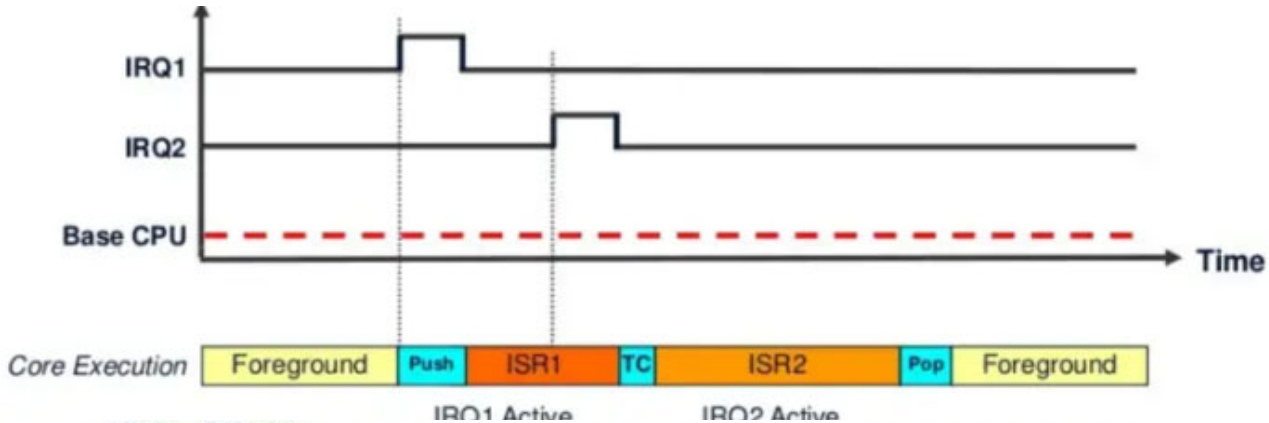
Need to send “hello” on serial port,
need to send “stop” on serial port

Without masking : `helstoplo`
With masking : `hellostop`

Interrupts : Nested interrupts

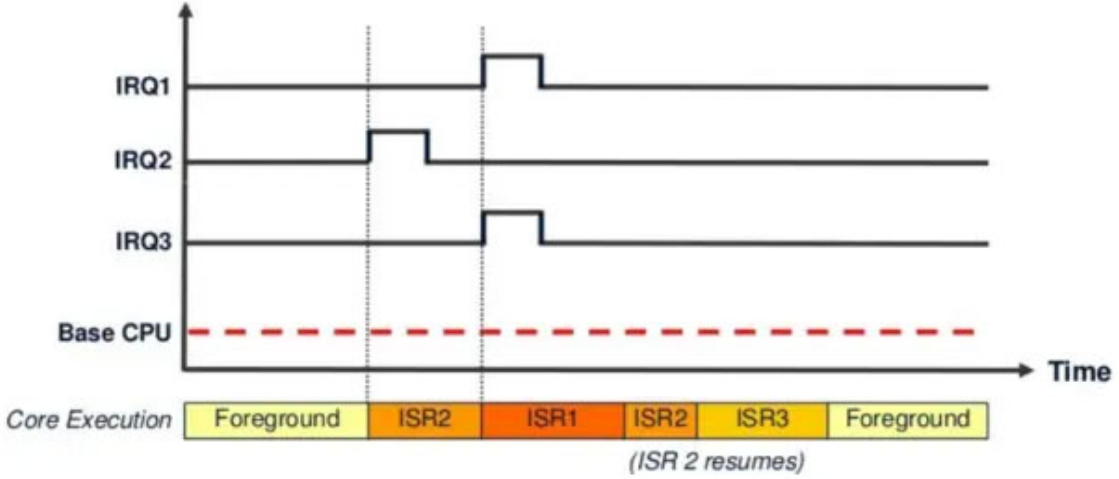
ISR1 > ISR2 > ISR3

Normal case



Higher Priority

Higher priority case



Interrupts : Nested interrupts

Hes·so VALAIS WALLIS

Remember the same hardware

With nested interrupts, the same problem can append like with main (for example int1 and int2) !

int2 has a higher priority than int1 : `helstoplo`

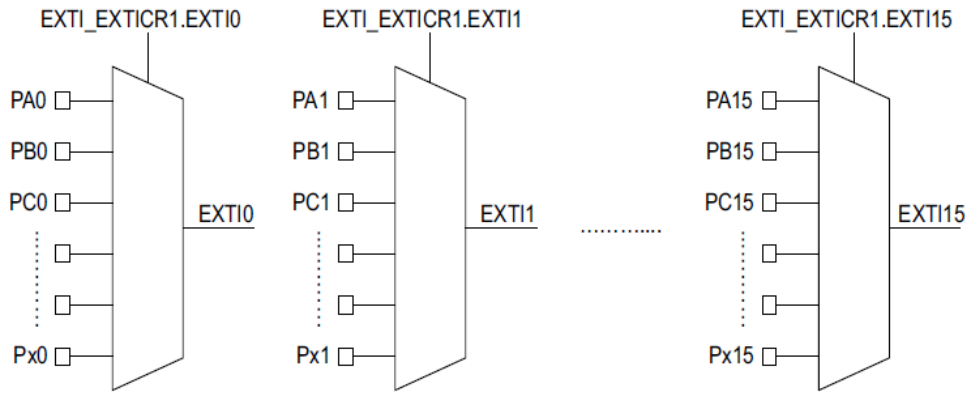
int2 has a lower priority than int1 : `sthelloop`

The solution is: same priority level for both

Valais excellence

Interrupts : I/O ports interrupts

- **There is 16 external interrupts depending on the port pin number, caution when making schematic**



Interrupts : I/O ports interrupts

- **Configure GPIO input as interrupt**

```
...
/*Configure GPIO pins : S2_Pin PB15 */
__HAL_RCC_GPIOB_CLK_ENABLE();
GPIO_InitStruct.Pin = 0x8000;                  // pin 15 for example
GPIO_InitStruct.Mode = GPIO_MODE_IT_FALLING; // for example
GPIO_InitStruct.Pull = GPIO_PULLUP;            // for example
HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);       // init port
HAL_NVIC_SetPriority(EXTI15_IRQn, 3, 0);      // priority 3, no subpriority
HAL_NVIC_EnableIRQ(EXTI15_IRQn);               // enable interrupt
...
```

Interrupts : I/O ports interrupts

- Vector table (special address) and ISR code

```
...
.section .isr_vector,"a",%progbits
.type g_pfnVectors, %object
g_pfnVectors:
.word _estack
.word Reset_Handler
.word NMI_Handler
.word HardFault_Handler
.word MemManage_Handler
.word BusFault_Handler
.word UsageFault_Handler
.word SecureFault_Handler
.word 0
.word 0
.word 0
.word SVC_Handler
.word DebugMon_Handler
.word 0
.word PendSV_Handler
.word SysTick_Handler
.word WWDG_IRQHandler
.word PVD_PVM_IRQHandler
.word RTC_IRQHandler
.word RTC_S_IRQHandler
.word TAMP_IRQHandler
.word RAMCFG_IRQHandler
.word FLASH_IRQHandler
.word FLASH_S_IRQHandler
.word GTZC_IRQHandler
.word RCC_IRQHandler
.word RCC_S_IRQHandler
.word EXTI0_IRQHandler
.word EXTI1_IRQHandler
.word EXTI2_IRQHandler
...
```

```
void EXTI0_IRQHandler(void)
{
  /* USER CODE BEGIN EXTI0_IRQn 0 */
  /* USER CODE END EXTI0_IRQn 0 */
  HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_0);
  /* USER CODE BEGIN EXTI0_IRQn 1 */
  /* USER CODE END EXTI0_IRQn 1 */
}
```

stm32u5xx_it.c

```
void HAL_GPIO_EXTI_IRQHandler(uint16_t GPIO_Pin)
{
/* EXTI line interrupt detected */
  if (__HAL_GPIO_EXTI_GET_RISING_IT(GPIO_Pin) != 0U)
  {
    __HAL_GPIO_EXTI_CLEAR_RISING_IT(GPIO_Pin);
    HAL_GPIO_EXTI_Rising_Callback(GPIO_Pin);
  }


  if (__HAL_GPIO_EXTI_GET_FALLING_IT(GPIO_Pin) != 0U)
  {
    __HAL_GPIO_EXTI_CLEAR_FALLING_IT(GPIO_Pin);
    HAL_GPIO_EXTI_Falling_Callback(GPIO_Pin);
  }
}
```

stm32u5xx_hal_gpio.c

Interrupts : I/O ports interrupts

- **Callback function**

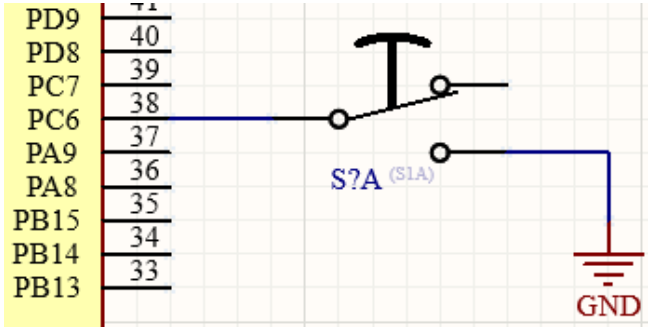
```
__weak void HAL_GPIO_EXTI_Falling_Callback(uint16_t GPIO_Pin)
{
/* Prevent unused argument(s) compilation warning */
UNUSED(GPIO_Pin);
/* NOTE: This function should not be modified, when the callback is
needed,
the HAL_GPIO_EXTI_Falling_Callback could be implemented in the user
file
*/
}
```

stm32u5xx_hal_gpio.c

You can do what you want in the interrupt function but never spend too much time

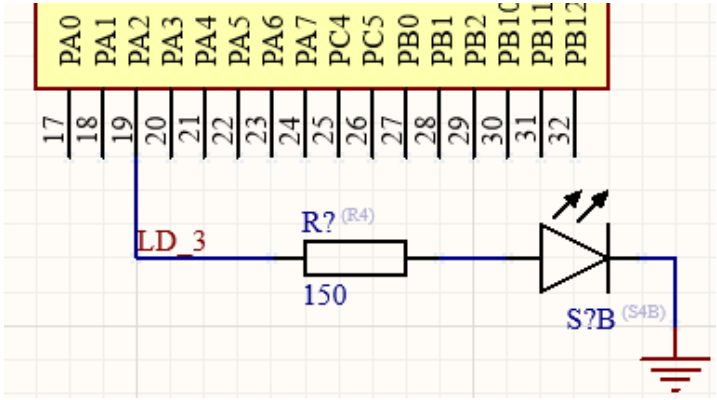
Interrupts : Exercise 1/2

Write the code to enable interrupt on pin GPIOC 6 to react when a button is pressed and the priority will be 5



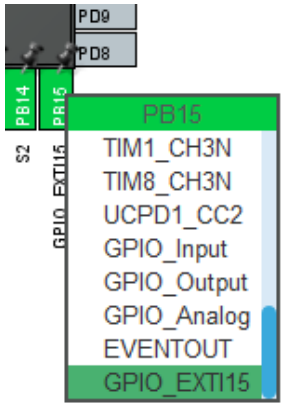
Interrupts : Exercise 2/2

Write the callback function to turn on the light on the GPIOA 2



Interrupts : STM32 graphic implementation

Pinout view



GPIO view

PB15	n/a	n/a	Non-pr...	n/a	Extern...	No pul...	n/a	n/a		☐
PD2	n/a	n/a	n/a	Low	Outpu...	No pul...	Low	n/a	RED_...	☑
PD4	n/a	n/a	n/a	Low	Outpu...	No pul...	Low	n/a	GREE...	☑
PE4	n/a	n/a	n/a	High	Outpu...	No pul...	Low	n/a	LCD_ON	☑
PE5	n/a	n/a	Non-pr...	n/a	Extern...	No pul...	n/a	n/a	TP_IRQ	☑

PB15 Configuration :

Pin Privilege access	Non-privileged access
GPIO mode	External Interrupt Mode with Rising edge trigger detection
GPIO Pull-up/Pull-down	No pull-up and no pull-down

NVIC view

EXTI Line10 interrupt	☑	0	0
EXTI Line14 interrupt	☑	0	0
EXTI Line15 interrupt	☑	4	0
USART1 global interrupt	☐	0	0
FPU global interrupt	☐	0	0
Instruction cache global interrupt	☐	0	0

Interrupts : The already enabled interrupts

NVIC Interrupt Table	Enabled	Preemption Priority
Non maskable interrupt	☑	0
Hard fault interrupt	☑	0
Memory management fault	☑	0
Prefetch fault, memory access fault	☑	0
Undefined instruction or illegal state	☑	0
System service call via SWI instruction	☑	0
Debug monitor	☑	0
Pendable request for system service	☑	0
Time base: System tick timer	☑	15

Reasons examples:

NMI: clock failure, flash error, …
Hard fault: illegal instruction, memory access out of bounds, …
Memory management: when memory protection used with OS
SWI: software call to change priority mechanism
Time base: the ticks (each millisecond)

Interrupts : multi-thread caution, again

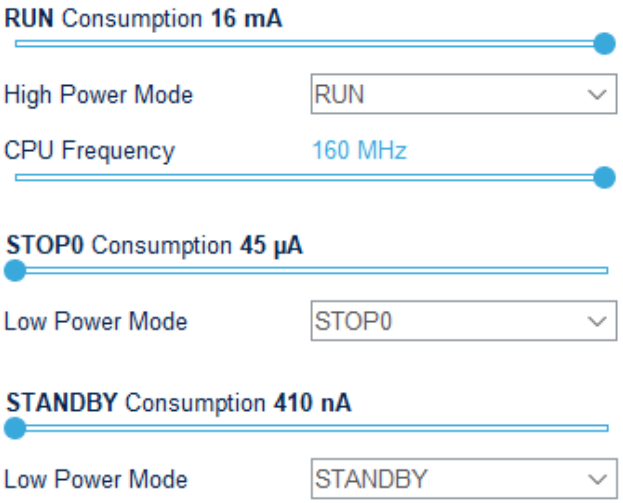
```
uint32_t myCounter;                          // global variable
void HAL_GPIO_EXTI_Falling_Callback (uint16_t GPIO_Pin)
{
  myCounter++;                              // increment global variable
}

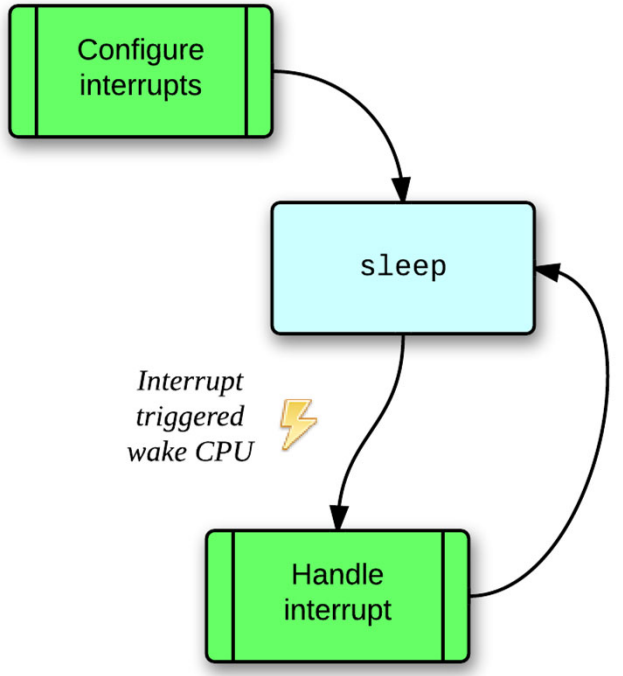
void main(void)
{
  // some code before
  while(1)
  {
    if(myCounter == 0x0001FFFF)     // test global variable
    {
     // some code here
    }
  }
}
```

Interrupts : sleep / stop / standby mode

microcontroller can be in a mode consuming less power like

- `HAL_PWR_EnterSLEEPMode()`
- `HAL_PWR_EnterSTOPMode()`
- `HAL_PWR_EnterSTANDBYMode()`





Interrupts : who wakes ?

A/D converter

USART RX

Interrupt pin

Timer

RTC

Sleep mode

External reset

Watch-dog timer

SysTick

